

WAN William
LO Hsiao-Wen-Paul
RAKOTOARIVELO Teddy
INGE1-APP-BDML
2025-2026

Projet de Base de Données (ALSI61)

Gestion d'un Système de Gestion de produits



Table des matières

I. Description du domaine.....	3
1. Choix du domaine.....	3
2. Règles métiers.....	3
3. Dictionnaire des données.....	4
II. Modélisation de la base de données : MCD et MLD.....	6
1. Modèle Conceptuel de Données (MCD).....	6
2. Modèle Logique de Données (MLD).....	6
III. Base de données SQL : Script DDL, DML et requêtes SQL.....	7
1. Script de création des tables.....	7
2. Script de création des triggers.....	8
3. Script de remplissage des tables.....	9
IV. Présentation de l'application.....	13

I. Description du domaine

1. Choix du domaine

J'ai choisi de modéliser un système de gestion de produits car il s'agit d'un domaine central dans le retail et le e-commerce, où une bonne gestion du catalogue, des variantes (taille, couleur) et des stocks est essentielle pour assurer les ventes et éviter les ruptures. Ce sujet est à la fois concret et réaliste, car on retrouve ces problématiques dans des entreprises comme Uniqlo, et il permet de travailler sur des aspects techniques intéressants comme la modélisation des données, les relations entre entités et les contraintes métier. De plus, il est particulièrement pertinent dans le contexte de la majeure, car il permet d'effectuer des analyses avancées (prévision des ventes, analyse des tendances). Enfin, c'est un choix pertinent à la fois académiquement et professionnellement, car il démontre la capacité à concevoir un système complet, structuré et évolutif, directement applicable en entreprise.

2. Règles métiers

Catégories :

Une catégorie possède un identifiant unique.

Une catégorie doit avoir un nom obligatoire.

Une catégorie peut avoir au maximum une seule catégorie parente.

Une catégorie peut avoir plusieurs sous-catégories.

Si une catégorie parente est supprimée, ses sous-catégories deviennent sans parent.

Produits :

Un produit possède un identifiant unique.

Un produit doit avoir un nom obligatoire.

Un produit peut avoir une description.

Un produit peut avoir une catégorie.

Le prix d'un produit doit être défini et ne peut pas être nul.

Un produit n'est vendable que via ses SKU, même en taille unique (car étant le seul SKU).

La date de création d'un produit est automatiquement définie au jour courant.

Couleurs :

Une couleur possède un identifiant unique.

Une couleur doit avoir un nom obligatoire.

Le nom d'une couleur doit être unique.

Une couleur ne peut pas être supprimée si elle est utilisée par un SKU.

Tailles :

Une taille possède un identifiant unique.

Une taille doit avoir un nom obligatoire.

Le nom d'une taille doit être unique.

Une taille ne peut pas être supprimée si elle est utilisée par un SKU.

SKU (variantes produit) :

Un SKU possède un identifiant unique.

Un SKU appartient obligatoirement à un seul produit.

Un SKU doit être associé à une taille existante.

Un SKU doit être associé à une couleur existante.
Une combinaison produit + taille + couleur doit être unique.
Si un produit est supprimé, ses SKU sont automatiquement supprimés.

Localisations :

Une localisation possède un identifiant unique.
Une localisation doit avoir un nom obligatoire.
Une localisation doit être de type STORE, WAREHOUSE ou OUTLET.
Une localisation peut contenir plusieurs stocks.

Stock :

Une ligne de stock correspond à un SKU dans une localisation.
Un SKU ne peut apparaître qu'une seule fois par localisation.
La quantité de stock doit être définie.
La quantité de stock ne doit pas être négative (règle métier implicite).
Si un SKU est supprimé, son stock est supprimé.
Si une localisation est supprimée, son stock est supprimé.

Clients :

Un client possède un identifiant unique.
Un client doit avoir un prénom et un nom.
L'email d'un client est obligatoire et unique.

Commandes :

Une commande appartient à un seul client.
Une commande est associée à une seule localisation.
Une commande doit avoir un statut parmi PENDING, PAID ou CANCELLED.
La date de commande est automatiquement définie.
Si un client est supprimé, ses commandes sont supprimées.
Une localisation utilisée dans une commande ne peut pas être supprimée.

Lignes de commande :

Une ligne de commande appartient à une seule commande.
Une ligne de commande concerne un seul SKU.
Une ligne de commande doit avoir une quantité définie.
Une ligne de commande doit avoir un prix défini.
Si une commande est supprimée, ses lignes sont supprimées.
Un SKU utilisé dans une commande ne peut pas être supprimé.

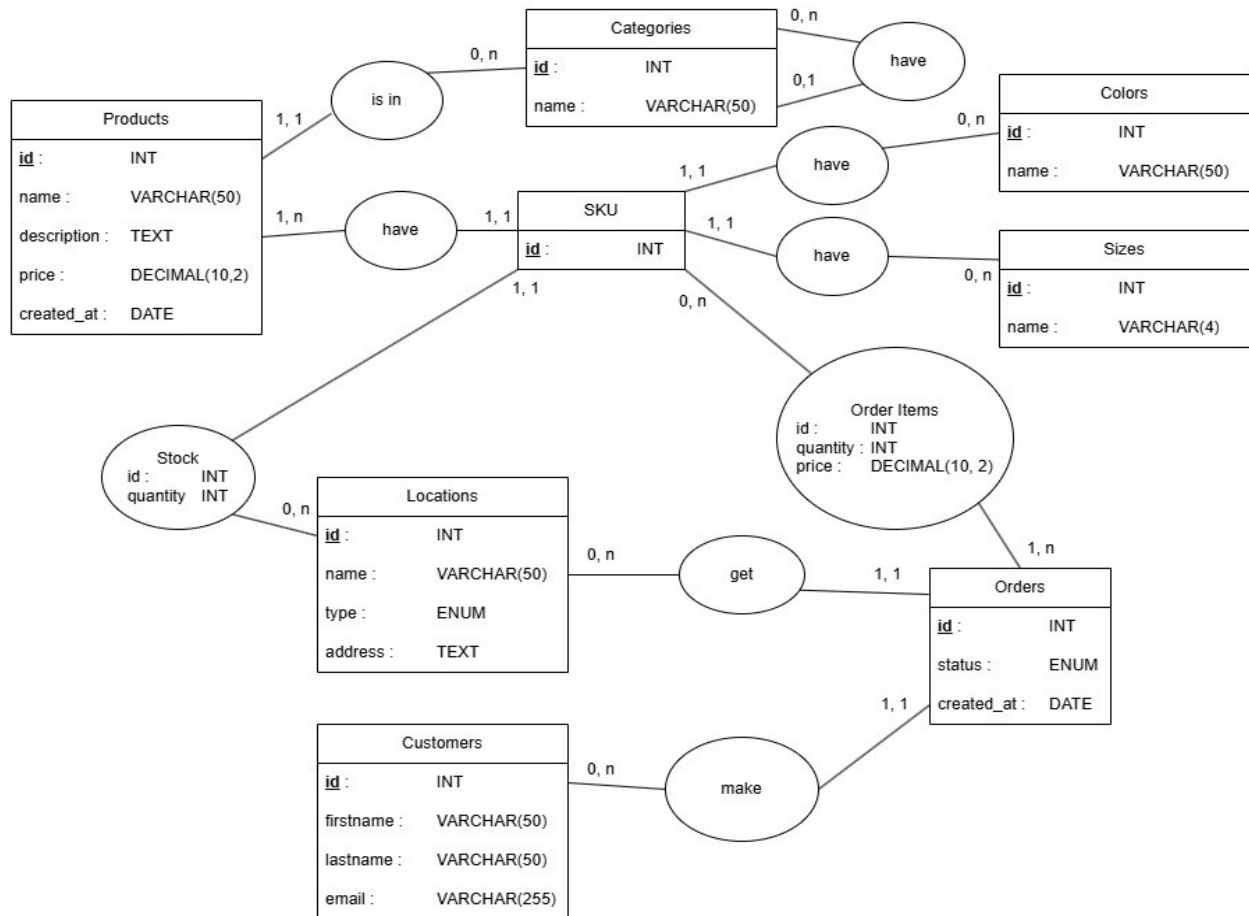
3. Dictionnaire des données

Table	Attribut	Type SQL	Contraintes	Description
categories	id	INT	PK, AUTO_INCREMENT	Identifiant unique de la catégorie
categories	name	VARCHAR(50)	NOT NULL	Nom de la catégorie
categories	parent_id	INT	FK → categories(id), NULL autorisé	Catégorie parente
products	id	INT	PK, AUTO_INCREMENT	Identifiant unique du produit
products	category_id	INT	FK → categories(id), NULL autorisé	Catégorie du produit

products	name	VARCHAR(50)	NOT NULL	Nom du produit
products	description	TEXT	NULL autorisé	Description du produit
products	price	DECIMAL(10,2)	NOT NULL, DEFAULT 0	Prix du produit
products	created_at	DATE	DEFAULT CURRENT_DATE	Date de création du produit
colors	id	INT	PK, AUTO_INCREMENT	Identifiant unique de la couleur
colors	name	VARCHAR(50)	NOT NULL, UNIQUE	Nom unique de la couleur
sizes	id	INT	PK, AUTO_INCREMENT	Identifiant unique de la taille
sizes	name	VARCHAR(4)	NOT NULL, UNIQUE	Nom unique de la taille
sku	id	INT	PK, AUTO_INCREMENT	Identifiant du SKU
sku	product_id	INT	NOT NULL, FK → products(id), ON DELETE CASCADE	Produit associé
sku	size_id	INT	NOT NULL, FK → sizes(id), ON DELETE RESTRICT	Taille associée
sku	color_id	INT	NOT NULL, FK → colors(id), ON DELETE RESTRICT	Couleur associée
locations	id	INT	PK, AUTO_INCREMENT	Identifiant de la localisation
locations	name	VARCHAR(50)	NOT NULL	Nom de la localisation
locations	type	ENUM	NOT NULL	Type (STORE, WAREHOUSE, OUTLET)
locations	address	TEXT	NULL autorisé	Adresse
stock	id	INT	PK, AUTO_INCREMENT	Identifiant du stock
stock	sku_id	INT	NOT NULL, FK → sku(id), ON DELETE CASCADE	SKU concerné
stock	location_id	INT	NOT NULL, FK → locations(id), ON DELETE CASCADE	Localisation
stock	quantity	INT	NOT NULL, DEFAULT 0	Quantité en stock
customers	id	INT	PK, AUTO_INCREMENT	Identifiant du client
customers	firstname	VARCHAR(50)	NOT NULL	Prénom du client
customers	lastname	VARCHAR(50)	NOT NULL	Nom du client
customers	email	VARCHAR(255)	NOT NULL, UNIQUE	Email unique
orders	id	INT	PK, AUTO_INCREMENT	Identifiant de la commande
orders	customer_id	INT	NOT NULL, FK → customers(id), ON DELETE CASCADE	Client
orders	location_id	INT	NOT NULL, FK → locations(id), ON DELETE RESTRICT	Localisation
orders	status	ENUM	NOT NULL	Statut (PENDING, PAID, CANCELLED)
orders	created_at	DATE	DEFAULT CURRENT_DATE	Date de commande
order_items	id	INT	PK, AUTO_INCREMENT	Identifiant de la ligne de commande
order_items	order_id	INT	NOT NULL, FK → orders(id), ON DELETE CASCADE	Commande associée
order_items	sku_id	INT	NOT NULL, FK → sku(id), ON DELETE RESTRICT	SKU commandé
order_items	quantity	INT	NOT NULL, DEFAULT 0	Quantité
order_items	price	DECIMAL(10,2)	NOT NULL	Prix unitaire

II. Modélisation de la base de données : MCD et MLD

1. Modèle Conceptuel de Données (MCD)



2. Modèle Logique de Données (MLD)

Categories(id, name, #parent_id)
Products(id, name, description, price, created_at, #category_id)
Colors(id, name)
Sizes(id, name)
SKU(id, #product_id, #size_id, #color_id)
Locations(id, name, type, address)
Customers(id, firstname, lastname, email)
Orders(id, status, created_at, #customer_id, #location_id)
Stock(id, quantity, #location_id, #sku_id)
Order_Items(id, quantity, price, #order_id, #sku_id)

Des identifiants techniques ont été utilisés afin de simplifier les relations entre les tables et éviter l'utilisation de clés composées. Les contraintes d'unicité permettent néanmoins de garantir le respect des règles établies (SKU, Stock, Order_Items).

III. Base de données SQL : Script DDL, DML et requêtes SQL

1. Script de création des tables

```
CREATE DATABASE IF NOT EXISTS db_efrei_project;
USE db_efrei_project;

SET FOREIGN_KEY_CHECKS = 0;

DROP TABLE IF EXISTS order_items;
DROP TABLE IF EXISTS stock;
DROP TABLE IF EXISTS orders;
DROP TABLE IF EXISTS sku;
DROP TABLE IF EXISTS products;
DROP TABLE IF EXISTS categories;
DROP TABLE IF EXISTS customers;
DROP TABLE IF EXISTS locations;
DROP TABLE IF EXISTS colors;
DROP TABLE IF EXISTS sizes;

SET FOREIGN_KEY_CHECKS = 1;

CREATE TABLE categories (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
  parent_id INT NULL,
  CONSTRAINT fk_category_parent
    FOREIGN KEY (parent_id) REFERENCES categories(id)
    ON DELETE SET NULL
) ENGINE=InnoDB;

CREATE TABLE products (
  id INT AUTO_INCREMENT PRIMARY KEY,
  category_id INT NULL,
  name VARCHAR(50) NOT NULL,
  description TEXT,
  price DECIMAL(10,2) NOT NULL DEFAULT 0,
  created_at DATE DEFAULT (CURRENT_DATE),
  CONSTRAINT fk_product_category
    FOREIGN KEY (category_id) REFERENCES categories(id)
    ON DELETE RESTRICT
) ENGINE=InnoDB;

CREATE TABLE colors (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(50) NOT NULL UNIQUE
) ENGINE=InnoDB;

CREATE TABLE sizes (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(4) NOT NULL UNIQUE
```

```
) ENGINE=InnoDB;
```

```
CREATE TABLE sku (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  product_id INT NOT NULL,  
  size_id INT NOT NULL,  
  color_id INT NOT NULL,  
  UNIQUE (product_id, size_id, color_id),  
  CONSTRAINT fk_sku_product  
    FOREIGN KEY (product_id) REFERENCES products(id)  
    ON DELETE CASCADE,  
  CONSTRAINT fk_sku_size  
    FOREIGN KEY (size_id) REFERENCES sizes(id)  
    ON DELETE RESTRICT,  
  CONSTRAINT fk_sku_color  
    FOREIGN KEY (color_id) REFERENCES colors(id)  
    ON DELETE RESTRICT  
) ENGINE=InnoDB;
```

```
CREATE TABLE locations (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(50) NOT NULL,  
  type ENUM('STORE', 'WAREHOUSE', 'OUTLET') NOT NULL,  
  address TEXT  
) ENGINE=InnoDB;
```

```
CREATE TABLE stock (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  location_id INT NOT NULL,  
  sku_id INT NOT NULL,  
  quantity INT NOT NULL DEFAULT 0,  
  UNIQUE (location_id, sku_id),  
  CHECK (quantity >= 0),  
  CONSTRAINT fk_stock_location  
    FOREIGN KEY (location_id) REFERENCES locations(id)  
    ON DELETE CASCADE,  
  CONSTRAINT fk_stock_sku  
    FOREIGN KEY (sku_id) REFERENCES sku(id)  
    ON DELETE CASCADE  
) ENGINE=InnoDB;
```

```
CREATE TABLE customers (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  firstname VARCHAR(50) NOT NULL,  
  lastname VARCHAR(50) NOT NULL,  
  email VARCHAR(255) NOT NULL UNIQUE  
) ENGINE=InnoDB;
```

```
CREATE TABLE orders (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  customer_id INT NOT NULL,
```



```
location_id INT NOT NULL,  
status ENUM('PENDING','PAID','CANCELLED') NOT NULL,  
created_at DATE DEFAULT (CURRENT_DATE),  
CONSTRAINT fk_order_customer  
    FOREIGN KEY (customer_id) REFERENCES customers(id)  
    ON DELETE CASCADE,  
CONSTRAINT fk_order_location  
    FOREIGN KEY (location_id) REFERENCES locations(id)  
    ON DELETE RESTRICT  
) ENGINE=InnoDB;  
  
CREATE TABLE order_items (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    order_id INT NOT NULL,  
    sku_id INT NOT NULL,  
    quantity INT NOT NULL DEFAULT 0,  
    price DECIMAL(10,2) NOT NULL,  
    UNIQUE (order_id, sku_id),  
    CHECK (quantity >= 0),  
    CONSTRAINT fk_orderitem_order  
        FOREIGN KEY (order_id) REFERENCES orders(id)  
        ON DELETE CASCADE,  
    CONSTRAINT fk_orderitem_sku  
        FOREIGN KEY (sku_id) REFERENCES sku(id)  
        ON DELETE RESTRICT  
) ENGINE=InnoDB;
```

2. Script de création des triggers

```
DELIMITER //
```

```
-- Trigger 1 : empêcher un stock négatif
CREATE TRIGGER block_negative_stock
BEFORE UPDATE ON stock
FOR EACH ROW
BEGIN
    IF NEW.quantity < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Le stock ne peut pas etre negatif';
    END IF;
END //
```

```
-- Trigger 2 : affecter automatiquement le prix du produit à la ligne de commande
CREATE TRIGGER set_price_order_item
BEFORE INSERT ON order_items
FOR EACH ROW
BEGIN
    DECLARE v_price DECIMAL(10,2);
    SELECT p.price INTO v_price
    FROM products p
    INNER JOIN sku s ON s.product_id = p.id
    WHERE s.id = NEW.sku_id;
    SET NEW.price = v_price;
END //
```

```
-- Trigger 3 : vérifier le stock avant d'ajouter une ligne de commande
CREATE TRIGGER check_stock_before_order
BEFORE INSERT ON order_items
FOR EACH ROW
FOLLOWS set_price_order_item
BEGIN
    DECLARE v_location_id INT;
    DECLARE v_stock INT DEFAULT 0;

    SELECT location_id INTO v_location_id
    FROM orders
    WHERE id = NEW.order_id;

    SELECT COALESCE(SUM(quantity), 0) INTO v_stock
    FROM stock
    WHERE location_id = v_location_id AND sku_id = NEW.sku_id;

    IF v_stock < NEW.quantity THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Stock insuffisant pour cette commande';
    END IF;
END //
```

```

-- Trigger 4 : décrémenter le stock quand une commande passe à PAID
CREATE TRIGGER update_stock_after_payment
AFTER UPDATE ON orders
FOR EACH ROW
BEGIN
    IF NEW.status = 'PAID' AND OLD.status != 'PAID' THEN
        UPDATE stock s
        INNER JOIN order_items oi ON s.sku_id = oi.sku_id
        SET s.quantity = s.quantity - oi.quantity
        WHERE oi.order_id = NEW.id AND s.location_id = NEW.location_id;
    END IF;
END //

-- Fonction 1 : total d'une commande
CREATE FUNCTION get_order_total(p_order_id INT)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE v_total DECIMAL(10,2);
    SELECT COALESCE(SUM(quantity * price), 0) INTO v_total
    FROM order_items
    WHERE order_id = p_order_id;
    RETURN v_total;
END //

-- Fonction 2 : stock total d'un SKU (toutes locations)
CREATE FUNCTION get_total_stock(p_sku_id INT)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE v_total INT;
    SELECT COALESCE(SUM(quantity), 0) INTO v_total
    FROM stock
    WHERE sku_id = p_sku_id;
    RETURN v_total;
END //

DELIMITER ;

```

3. Script de remplissage des tables

```
INSERT INTO categories (id, name, parent_id)
VALUES (1, 'Hauts', NULL),
       (2, 'Bas', NULL),
       (3, 'Chaussures', NULL),
       (4, 'Accessoires', NULL),
       (5, 'T-Shirts', 1),
       (6, 'Chemises', 1),
       (7, 'Jeans', 2),
       (8, 'Pantalons', 2),
       (9, 'Sneakers', 3);
-- Produits (22)
INSERT INTO products (id, category_id, name, description, price)
VALUES (
    1,
    5,
    'T-Shirt Col Rond',
    'T-shirt basique en coton a col rond',
    14.90
),
(
    2,
    5,
    'T-Shirt Col V',
    'T-shirt coupe classique col V',
    14.90
),
(
    3,
    5,
    'T-Shirt Oversize',
    'T-shirt coupe ample tendance',
    19.90
),
(
    4,
    5,
    'T-Shirt Dry-EX',
    'T-shirt technique respirant',
    24.90
),
(
    5,
    5,
    'T-Shirt Raye',
    'T-shirt marinier a rayures',
    17.90
),
(
    6,
```

```
5,  
'T-Shirt Poche',  
'T-shirt avec poche poitrine',  
15.90  
,  
(  
7,  
6,  
'Chemise Oxford',  
'Chemise en coton Oxford boutonne',  
29.90  
,  
(  
8,  
6,  
'Chemise Lin',  
'Chemise legere en lin naturel',  
34.90  
,  
(  
9,  
7,  
'Jean Slim',  
'Jean coupe slim stretch',  
39.90  
,  
(  
10,  
7,  
'Jean Regular',  
'Jean coupe droite classique',  
39.90  
,  
(  
11,  
7,  
'Jean Wide',  
'Jean coupe large tendance',  
44.90  
,  
(  
12,  
8,  
'Chino Slim',  
'Pantalon chino coupe ajustee',  
34.90  
,  
(  
13,  
8,  
'Chino Regular',
```

```
'Pantalon chino coupe droite',
34.90
),
(
14,
9,
'Sneakers Canvas',
'Baskets en toile legere',
29.90
),
(
15,
9,
'Sneakers Cuir',
'Baskets en cuir premium',
99.90
),
(
16,
9,
'Sneakers Running',
'Baskets de course amorties',
89.90
),
(
17,
3,
'Mocassins Cuir',
'Mocassins en cuir souple',
89.90
),
(
18,
4,
'Casquette Logo',
'Casquette avec logo brode',
14.90
),
(
19,
4,
'Ceinture Cuir',
'Ceinture en cuir veritable',
19.90
),
(
20,
4,
'Echarpe Laine',
'Echarpe en laine merinos',
24.90
```

```

),
(
    21,
    4,
    'Sac Tote',
    'Sac fourre-tout en coton',
    9.90
),
(
    22,
    4,
    'Bonnet Maille',
    'Bonnet en maille tricotée',
    12.90
);
-- Couleurs (5)
INSERT INTO colors (id, name)
VALUES (1, 'Noir'),
       (2, 'Blanc'),
       (3, 'Bleu'),
       (4, 'Rouge'),
       (5, 'Gris');
-- Tailles (4)
INSERT INTO sizes (id, name)
VALUES (1, 'XS'),
       (2, 'S'),
       (3, 'M'),
       (4, 'L');
-- SKU (60 combinaisons produit + taille + couleur)
INSERT INTO sku (id, product_id, size_id, color_id)
VALUES -- T-Shirt Col Rond (4 SKU)
       (1, 1, 1, 1),
       (2, 1, 2, 1),
       (3, 1, 3, 2),
       (4, 1, 4, 3),
       -- T-Shirt Col V (4 SKU)
       (5, 2, 2, 1),
       (6, 2, 3, 1),
       (7, 2, 3, 4),
       (8, 2, 4, 5),
       -- T-Shirt Oversize (3 SKU)
       (9, 3, 3, 1),
       (10, 3, 4, 1),
       (11, 3, 3, 2),
       -- T-Shirt Dry-EX (3 SKU)
       (12, 4, 1, 3),
       (13, 4, 2, 3),
       (14, 4, 3, 5),
       -- T-Shirt Rayé (2 SKU)
       (15, 5, 2, 4),
       (16, 5, 3, 2),

```

```
-- T-Shirt Poche (2 SKU)
(17, 6, 3, 1),
(18, 6, 4, 5),
-- Chemise Oxford (3 SKU)
(19, 7, 2, 2),
(20, 7, 3, 2),
(21, 7, 3, 3),
-- Chemise Lin (3 SKU)
(22, 8, 2, 2),
(23, 8, 3, 2),
(24, 8, 4, 5),
-- Jean Slim (4 SKU)
(25, 9, 2, 3),
(26, 9, 3, 3),
(27, 9, 3, 1),
(28, 9, 4, 3),
-- Jean Regular (3 SKU)
(29, 10, 2, 3),
(30, 10, 3, 3),
(31, 10, 4, 1),
-- Jean Wide (3 SKU)
(32, 11, 3, 3),
(33, 11, 4, 3),
(34, 11, 3, 1),
-- Chino Slim (3 SKU)
(35, 12, 2, 1),
(36, 12, 3, 5),
(37, 12, 4, 3),
-- Chino Regular (2 SKU)
(38, 13, 2, 1),
(39, 13, 3, 5),
-- Sneakers Canvas (3 SKU)
(40, 14, 2, 2),
(41, 14, 3, 1),
(42, 14, 4, 4),
-- Sneakers Cuir (3 SKU)
(43, 15, 2, 1),
(44, 15, 3, 1),
(45, 15, 4, 2),
-- Sneakers Running (3 SKU)
(46, 16, 2, 3),
(47, 16, 3, 5),
(48, 16, 4, 1),
-- Mocassins Cuir (2 SKU)
(49, 17, 3, 1),
(50, 17, 4, 1),
-- Casquette Logo (2 SKU)
(51, 18, 3, 1),
(52, 18, 3, 2),
-- Ceinture Cuir (2 SKU)
(53, 19, 3, 1),
```



```

(54, 19, 3, 5),
-- Echarpe Laine (2 SKU)
(55, 20, 3, 5),
(56, 20, 3, 4),
-- Sac Tote (2 SKU)
(57, 21, 3, 2),
(58, 21, 3, 1),
-- Bonnet Maille (2 SKU)
(59, 22, 3, 5),
(60, 22, 3, 1);
-- Locations (4)
INSERT INTO locations (id, name, type, address)
VALUES (
    1,
    'Boutique Paris Opera',
    'STORE',
    '1 Boulevard Haussmann, 75009 Paris'
),
(
    2,
    'Boutique Lyon Part-Dieu',
    'STORE',
    '17 Rue du Docteur Bouchut, 69003 Lyon'
),
(
    3,
    'Entrepot Central Lille',
    'WAREHOUSE',
    'Zone Industrielle, 59000 Lille'
),
(
    4,
    'Outlet Marseille',
    'OUTLET',
    'Les Terrasses du Port, 13002 Marseille'
);
-- Stock (60 lignes)
INSERT INTO stock (location_id, sku_id, quantity)
VALUES -- Paris (20 entrées)
(1, 1, 25),
(1, 2, 30),
(1, 3, 20),
(1, 4, 15),
(1, 5, 18),
(1, 6, 22),
(1, 7, 12),
(1, 8, 10),
(1, 12, 14),
(1, 13, 16),
(1, 14, 20),
(1, 25, 15),

```

```
(1, 26, 18),
(1, 27, 12),
(1, 35, 10),
(1, 36, 15),
(1, 43, 8),
(1, 44, 10),
(1, 51, 20),
(1, 53, 25),
-- Lyon (15 entrées)
(2, 1, 15),
(2, 5, 10),
(2, 9, 20),
(2, 10, 12),
(2, 11, 18),
(2, 19, 14),
(2, 20, 10),
(2, 23, 12),
(2, 29, 20),
(2, 30, 15),
(2, 38, 10),
(2, 39, 12),
(2, 41, 8),
(2, 47, 10),
(2, 52, 15),
-- Entrepot Lille (15 entrées)
(3, 2, 50),
(3, 9, 40),
(3, 20, 35),
(3, 21, 30),
(3, 23, 25),
(3, 26, 40),
(3, 30, 35),
(3, 32, 30),
(3, 34, 25),
(3, 38, 20),
(3, 44, 35),
(3, 49, 15),
(3, 53, 30),
(3, 56, 20),
(3, 58, 25),
-- Outlet Marseille (10 entrées)
(4, 3, 12),
(4, 7, 8),
(4, 24, 10),
(4, 31, 15),
(4, 37, 10),
(4, 40, 12),
(4, 46, 15),
(4, 51, 10),
(4, 55, 18),
(4, 59, 10);
```

```

-- Clients (5)
INSERT INTO customers (id, firstname, lastname, email)
VALUES (1, 'Marie', 'Dupont', 'marie.dupont@email.fr'),
      (2, 'Jean', 'Martin', 'jean.martin@email.fr'),
      (
        3,
        'Sophie',
        'Bernard',
        'sophie.bernard@email.fr'
      ),
      (4, 'Lucas', 'Petit', 'lucas.petit@email.fr'),
      (5, 'Emma', 'Richard', 'emma.richard@email.fr');

-- Commandes (15) – toutes insérées en PENDING d'abord
INSERT INTO orders (id, customer_id, location_id, status, created_at)
VALUES (1, 1, 1, 'PENDING', '2024-01-15'),
      (2, 1, 2, 'PENDING', '2024-01-20'),
      (3, 2, 1, 'PENDING', '2024-02-05'),
      (4, 2, 3, 'PENDING', '2024-02-10'),
      (5, 3, 1, 'PENDING', '2024-02-14'),
      (6, 3, 2, 'PENDING', '2024-02-28'),
      (7, 3, 4, 'PENDING', '2024-03-05'),
      (8, 4, 1, 'PENDING', '2024-03-10'),
      (9, 4, 2, 'PENDING', '2024-03-15'),
      (10, 4, 3, 'PENDING', '2024-03-20'),
      (11, 5, 1, 'PENDING', '2024-04-01'),
      (12, 5, 2, 'PENDING', '2024-04-10'),
      (13, 5, 4, 'PENDING', '2024-04-15'),
      (14, 1, 3, 'PENDING', '2024-04-20'),
      (15, 2, 4, 'PENDING', '2024-05-01');

-- Lignes de commande (25) – le prix est auto-rempli par le trigger
set_price_order_item
INSERT INTO order_items (order_id, sku_id, quantity, price)
VALUES (1, 1, 2, 0),
      -- T-Shirt Col Rond XS Noir x2
      (1, 5, 1, 0),
      -- T-Shirt Col V S Noir x1
      (2, 9, 1, 0),
      -- T-Shirt Oversize M Noir x1
      (2, 19, 1, 0),
      -- Chemise Oxford S Blanc x1
      (3, 25, 1, 0),
      -- Jean Slim S Bleu x1
      (3, 2, 2, 0),
      -- T-Shirt Col Rond S Noir x2
      (4, 20, 1, 0),
      -- Chemise Oxford M Blanc x1
      (5, 43, 1, 0),
      -- Sneakers Cuir S Noir x1
      (5, 35, 1, 0),
      -- Chino Slim S Noir x1
      (6, 29, 2, 0),

```

```

-- Jean Regular S Bleu x2
(6, 41, 1, 0),
-- Sneakers Canvas M Noir x1
(7, 40, 1, 0),
-- Sneakers Canvas S Blanc x1
(7, 51, 2, 0),
-- Casquette Logo M Noir x2
(8, 6, 1, 0),
-- T-Shirt Col V M Noir x1
(9, 23, 1, 0),
-- Chemise Lin M Blanc x1
(10, 32, 1, 0),
-- Jean Wide M Bleu x1
(10, 38, 1, 0),
-- Chino Regular S Noir x1
(11, 3, 1, 0),
-- T-Shirt Col Rond M Blanc x1
(11, 14, 1, 0),
-- T-Shirt Dry-EX M Gris x1
(12, 30, 1, 0),
-- Jean Regular M Bleu x1
(13, 46, 1, 0),
-- Sneakers Running S Bleu x1
(13, 55, 2, 0),
-- Echarpe Laine M Gris x2
(14, 49, 1, 0),
-- Mocassins Cuir M Noir x1
(14, 34, 2, 0),
-- Jean Wide M Noir x2
(15, 59, 1, 0);
-- Bonnet Maille M Gris x1
-- =====
-- Mise à jour des statuts (déclenche update_stock_after_payment)
-- =====
-- Commandes payées → le trigger décrémente le stock automatiquement
UPDATE orders
SET status = 'PAID'
WHERE id IN (1, 3, 5, 7, 11, 12, 13);
-- Commandes annulées → aucun impact sur le stock
UPDATE orders
SET status = 'CANCELLED'
WHERE id IN (4, 8);
-- Commandes 2, 6, 9, 10, 14, 15 restent PENDING

```

4. Requêtes SQL

R1. Afficher la liste de tous les enregistrements de votre entité principale, triés par ordre alphabétique.

Approche : SELECT simple avec ORDER BY alphabétique sur name

```
SELECT *  
FROM products  
ORDER BY name;
```

id	category_id	name	description	price	created_at
22	4	Bonnet Maille	Bonnet en maille tricotée	12.90	2026-05-31
18	4	Casquette Logo	Casquette avec logo brodé	14.90	2026-05-31
19	4	Ceinture Cuir	Ceinture en cuir véritable	19.90	2026-05-31
8	6	Chemise Lin	Chemise légère en lin naturel	34.90	2026-05-31
7	6	Chemise Oxford	Chemise en coton Oxford boutonnée	29.90	2026-05-31
13	8	Chino Regular	Pantalon chino coupe droite	34.90	2026-05-31

R2. Afficher les enregistrements satisfaisant un critère numérique (ex : montant > X, score > Y, âge < Z).

Approche : filtrage avec WHERE price > 50

```
SELECT *  
FROM products  
WHERE price > 50;
```

id	category_id	name	description	price	created_at
15	9	Sneakers Cuir	Baskets en cuir premium	99.90	2026-05-31
16	9	Sneakers Running	Baskets de course amorties	89.90	2026-05-31
17	3	Mocassins Cuir	Mocassins en cuir souple	89.90	2026-05-31

R3. Afficher les enregistrements liés à un identifiant passé en paramètre (ex : tous les éléments d'une catégorie donnée).

Approche : filtrage direct par category_id

```
SELECT id, name, price  
FROM products  
WHERE category_id = 5;
```

id	name	price
1	T-Shirt Col Rond	14.90
2	T-Shirt Col V	14.90
3	T-Shirt Oversize	19.90
4	T-Shirt Dry-EX	24.90
5	T-Shirt Rayé	17.90
6	T-Shirt Poche	15.90

R4. Afficher les informations combinées de deux entités liées (INNER JOIN).

Approche : INNER JOIN donc seuls les produits ayant une catégorie

```
SELECT p.name AS product_name, s.id sku_id  
FROM products p  
INNER JOIN sku s ON p.id = s.product_id;
```

id	produit	price	categorie
22	Bonnet Maille	12.90	Accessoires
18	Casquette Logo	14.90	Accessoires
19	Ceinture Cuir	19.90	Accessoires
20	Echarpe Laine	24.90	Accessoires
21	Sac Tote	9.90	Accessoires
23	test s	1551.00	Bas
17	Mocassins Cuir	89.90	Chaussures
8	Chemise Lin	34.90	Chemises

R5. Afficher tous les enregistrements d'une entité, même s'ils n'ont pas de correspondance dans une autre (LEFT JOIN).

Approche : LEFT JOIN donc on conserve tous les produits, y compris ceux dont category_id serait NULL

```
SELECT p.id, p.name AS produit, p.price, c.name AS categorie  
FROM products p  
LEFT JOIN categories c ON p.category_id = c.id  
ORDER BY c.name, p.name;
```

id	produit	price	categorie
22	Bonnet Maille	12.90	Accessoires
18	Casquette Logo	14.90	Accessoires
19	Ceinture Cuir	19.90	Accessoires
20	Echarpe Laine	24.90	Accessoires
21	Sac Tote	9.90	Accessoires

R6. Afficher des informations agrégées combinant plusieurs tables (ex : total par catégorie avec le nom de la catégorie).

Approche : GROUP BY catégorie + SUM(price) + JOIN pour le nom

```
SELECT c.name AS categorie, SUM(p.price) AS total_prix
FROM products p
INNER JOIN categories c ON p.category_id = c.id
GROUP BY c.id, c.name
ORDER BY total_prix DESC;
```

categorie	total_prix
Bas	1551.00
Sneakers	219.70
Jeans	124.70
T-Shirts	108.40
Chaussures	89.90
Accessoires	82.50
Pantalons	69.80
Chemises	64.80

R7. Afficher le nombre d'enregistrements par catégorie, trié par ordre décroissant.

Approche : LEFT JOIN pour inclure les catégories vides, COUNT sur products.id, ORDER BY DESC

```
SELECT c.name AS categorie, COUNT(p.id) AS nb_produits
FROM categories c
LEFT JOIN products p ON c.id = p.category_id
GROUP BY c.id, c.name
ORDER BY nb_produits DESC;
```

categorie	nb_produits
T-Shirts	6
Accessoires	5
Jeans	3
Sneakers	3

R8. Afficher les entités ayant un agrégat supérieur à un seuil donné (ex : plus de N éléments associés).

Approche : GROUP BY + HAVING COUNT > 5 pour filtrer après agrégation (T-Shirts = 6 produits)

```
SELECT c.name AS categorie, COUNT(p.id) AS nb_produits
FROM categories c
INNER JOIN products p ON c.id = p.category_id
GROUP BY c.id, c.name
HAVING COUNT(p.id) > 5;
```

categorie	nb_produits
T-Shirts	6

R9. Calculer une moyenne ou une somme par groupe, en filtrant avec HAVING.

Approche : GROUP BY + SUM + HAVING SUM > 200 (Sneakers = 219.70 €)

```
SELECT c.name AS categorie, SUM(p.price) AS total_prix
FROM products p
INNER JOIN categories c ON p.category_id = c.id
GROUP BY c.id, c.name
HAVING SUM(p.price) > 200;
```

categorie	total_prix
Sneakers	219.70

R10. Afficher le maximum ou le minimum d'un attribut numérique par groupe.

Approche : GROUP BY catégorie + MAX(price)

```
SELECT c.name AS categorie, MAX(p.price) AS prix_max
FROM products p
INNER JOIN categories c ON p.category_id = c.id
GROUP BY c.id, c.name
ORDER BY prix_max DESC;
```

R11. Afficher les enregistrements dont une valeur est supérieure à la moyenne globale (sousrequête scalaire).

Approche : sous-requête scalaire SELECT AVG(price) dans le WHERE

```
SELECT p.name AS produit, p.price, c.name AS categorie
FROM products p
INNER JOIN categories c ON p.category_id = c.id
WHERE p.price > (SELECT AVG(price) FROM products)
ORDER BY p.price DESC;
```

produit	price	categorie
Sneakers Cuir	99.90	Sneakers
Sneakers Running	89.90	Sneakers
Mocassins Cuir	89.90	Chaussures
Jean Wide	44.90	Jeans
Jean Slim	39.90	Jeans
Jean Regular	39.90	Jeans
Chemise Lin	34.90	Chemises
Chino Slim	34.90	Pantalons
Chino Regular	34.90	Pantalons

R12. Afficher les entités qui ont satisfait une condition sur tous leurs éléments associés (ALL ou NOT EXISTS).

Approche : NOT EXISTS avec sous-requête corrélée cherchant une commande NON payée pour ce client. " $\forall$ commandes PAID" $\Leftrightarrow$ " $\neg \exists$ commande non-PAID"

```
SELECT cu.id, cu.firstname, cu.lastname, cu.email
FROM customers cu
WHERE NOT EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = cu.id
    AND o.status != 'PAID'
);
```

id	firstname	lastname	email
5	Emma	Richard	emma.richard@email.fr

R13. Afficher un classement avec départage sur plusieurs critères (ORDER BY multicolonne ou RANK()).

Approche : fonction fenêtré RANK() partitionnée par category_id, ordonnée par price DESC

```
SELECT c.name AS categorie, p.name AS produit, p.price,
    RANK() OVER (
        PARTITION BY p.category_id
        ORDER BY p.price DESC
    ) AS rang
FROM products p
INNER JOIN categories c ON p.category_id = c.id
ORDER BY c.name, rang;
```

categorie	produit	price	rang
Accessoires	Echarpe Laine	24.90	1
Accessoires	Ceinture Cuir	19.90	2
Accessoires	Casquette Logo	14.90	3
Accessoires	Bonnet Maille	12.90	4
Accessoires	Sac Tote	9.90	5
Chaussures	Mocassins Cuir	89.90	1
Chemises	Chemise Lin	34.90	1
Chemises	Chemise Oxford	29.90	2
Jeans	Jean Wide	44.90	1

R14. Afficher les entités ayant participé à au moins deux catégories différentes (sous-requête avec COUNT DISTINCT).

Approche : GROUP BY sku_id sur la table stock, HAVING COUNT(DISTINCT location_id) >= 2

```
SELECT sk.id AS sku_id, p.name AS produit, sz.name AS taille, co.name AS couleur,
COUNT(DISTINCT st.location_id) AS nb_locations
FROM sku sk
INNER JOIN stock st ON sk.id = st.sku_id
INNER JOIN products p ON sk.product_id = p.id
INNER JOIN sizes sz ON sk.size_id = sz.id
INNER JOIN colors co ON sk.color_id = co.id
GROUP BY sk.id, p.name, sz.name, co.name
HAVING COUNT(DISTINCT st.location_id) >= 2
ORDER BY nb_locations DESC, p.name;
```

sku_id	produit	taille	couleur	nb_locations
51	Casquette Logo	M	Noir	2
53	Ceinture Cuir	M	Noir	2
23	Chemise Lin	M	Blanc	2
20	Chemise Oxford	M	Blanc	2
38	Chino Regular	S	Noir	2
30	Jean Regular	M	Bleu	2
26	Jean Slim	M	Bleu	2
44	Sneakers Cuir	M	Noir	2
1	T-Shirt Col Rond	XS	Noir	2

R15. Afficher, pour chaque groupe, l'élément ayant la valeur maximale d'un attribut. En cas d'égalité, afficher les deux.

Approche : sous-requête corrélée, donc on compare le prix de chaque produit au MAX(price) de sa propre catégorie

```
SELECT p.name AS produit, p.price, c.name AS categorie
FROM products p
INNER JOIN categories c ON p.category_id = c.id
WHERE p.price = (
    SELECT MAX(p2.price)
    FROM products p2
    WHERE p2.category_id = p.category_id
)
ORDER BY c.name, p.name;
```

produit	price	categorie
Echarpe Laine	24.90	Accessoires
Mocassins Cuir	89.90	Chaussures
Chemise Lin	34.90	Chemises
Jean Wide	44.90	Jeans
Chino Regular	34.90	Pantalons
Chino Slim	34.90	Pantalons
Sneakers Cuir	99.90	Sneakers
T-Shirt Dry-EX	24.90	T-Shirts

